

세르파엿 멀티 에이전트 AI 코치: 2025년 프로덕션 아키텍처 및 구현 청사진

서문: 개념적 아키텍처

1

제공된 AI_ANALYSIS_SYSTEM_ARCHITECTURE.pdf¹ 문서는 세르파엿의 AI 코칭 기능을 단일 에이전트에서 멀티 에이전트 시스템으로 전환하기 위한 탁월한 개념적 기반(Conceptual Foundation)을 제시합니다.¹ 본 보고서는 이 개념적 설계를 2025년 11월 SOTA(State-of-the-Art) 기술 스택을 사용하여 실제 프로덕션 환경에 배포할 수 있는 구체적이고 실행 가능한 기술 청사진(Technical Blueprint)으로 변환하는 것을 목표로 합니다.

현재 단일 에이전트 시스템은 15-20초에 달하는 긴 지연 시간(Long Latency), 단일 실패 지점(All-or-Nothing), 그리고 진행 상황을 알 수 없는 '블랙박스' UX라는 명확한 한계를 가지고 있습니다.¹

귀사가¹ 14-18페이지에 제시한 3-에이전트 아키텍처('운동 전문가', '감성 코치', '리더')는 전문성을 분리하고, 지연 시간을 7-10초로 단축하며, 'Progressive 3-Stage Display'를 통해 UX를 획기적으로 개선하는 것을 목표로 합니다.¹

본 보고서는¹ 19-31페이지에 기술된 개념적 Orchestrator 및 Agent 코드를 실제 프로덕션급 프레임워크와 데이터베이스(LangGraph, Neo4j)로 대체합니다. 또한,¹ 24-25페이지의 'Agent-Level Fallback Strategy'와¹ 25-27페이지의 'Transition Roadmap'을 2025년 최신 기술에 맞게 구체화하여, 귀사의 엔지니어링팀이 즉시 개발에 착수할 수 있는 상세 가이드를 제공합니다.

I. 2025년 멀티 에이전트 시스템을 위한 핵심 기술 스택 권장

귀사의 3-에이전트 설계를 성공적으로 구현하기 위해, 2025년 11월 현재 가장 성능과 안정성이 검증된 기술 스택을 선정합니다.

I.A. 오케스트레이션 프레임워크: LangGraph (CrewAI 대안 검토)

귀사의 아키텍처¹는 '운동 전문가' $\rightarrow$ '감성 코치' $\rightarrow$ '리더'로 이어지는 명확한 순차적(Sequential) 워크플로우를 요구합니다. 2025년 현재, 이 분야의 주요 도구는 LangGraph와 CrewAI입니다.²

CrewAI는 역할 기반(Role-based)이며 Process.sequential을 통해 순차 워크플로우를 쉽게 정의할 수 있어 ¹의 3-에이전트 '팀' 개념에 표면적으로 적합해 보입니다.⁶

하지만, 귀사의 아키텍처에서 가장 중요한 기술적 요구 사항은 ¹ 24-25페이지에 명시된 ****Agent-Level Fallback Strategy****입니다. 이 전략은 "IF '운동 전문가 에이전트'가 실패하면, THEN '기본 통계'를 반환하고, '감성 코치'는 '일반 격려'를 반환하라"는 복잡한 ****조건부 분기(Conditional Branching)****를 요구합니다.¹

CrewAI는 고수준 추상화(High-level Abstraction)에 중점을 두어, 이처럼 에이전트 실행 중간에 발생하는 예외(Exception)를 감지하고 워크플로우의 경로 자체를 동적으로 변경(Rerouting)하는 세밀한 제어가 복잡합니다.²

반면 LangGraph는 상태 머신(State Machine)을 기반으로 하며 ⁴, 이러한 시나리오를 위해 설계되었습니다. LangGraph의 '조건부 엣지(Conditional Edges)' ¹⁰ 기능을 사용하면, '운동 전문가' 노드(Node) 실행 후, "IF state.error is True, THEN route to fallback_expert_node"와 같은 로직을 완벽하게 구현할 수 있습니다.¹³

또한, ¹ 23페이지의 'Progressive 3-Stage Display' UX 요구 사항은 LangGraph의 stream() 또는 astream_events() ¹⁴를 통해 각 노드(에이전트)가 완료될 때마다 UI로 이벤트를 전송하는 기능과 정확히 일치합니다.

따라서 귀사의 'Fallback Strategy'와 'Progressive UX'라는 두 가지 핵심 요구 사항을 충족하기 위해서는 LangGraph가 기술적으로 월등히 우수한 선택입니다.

표 1: 오케스트레이션 프레임워크 비교 (2025년 11월 기준)

기능	CrewAI	LangGraph	세르파엠플을 위한 권장
----	--------	-----------	--------------

순차 워크플로우 ¹	우수 (핵심 기능). Process.sequential 로 간단히 정의.	우수. add_edge를 통해 명시적으로 순차 그래프 구성.	LangGraph
상태 관리 ¹	양호. 태스크 출력을 다음 태스크 컨텍스트로 전달.	탁월 (핵심 기능). 명시적인 StateGraph ⁹ 로 중앙 상태 관리.	LangGraph
Fallback 및 제어 ¹	제한적. 고수준 추상화로 인해 세밀한 예외 처리 및 동적 라우팅이 복잡함.	탁월 (핵심 기능). '조건부 엣지' ¹⁰ 가 이 시나리오를 위해 설계됨.	LangGraph
Progressive UX ¹	제한적. 전체 'Crew' 실행 완료 후 결과 반환이 기본.	탁월 (핵심 기능). stream/astream_ev ents로 각 노드 완료 시 실시간 피드백 가능.	LangGraph

I.B. 장기 기억(Long-Term Memory): Neo4j (GraphRAG + TKG)

귀사의 시스템은 사용자의 GlobalUser, GoalModel, RoutineModel ¹을 저장하고, 이를 RunningRecord ¹와 연결해야 합니다. Pinecone, Milvus 등 단순한 벡터 DB는 이 요구 사항을 온전히 충족하지 못합니다.¹⁷

단순 벡터 DB는 '10km 완주' ¹ 같은 텍스트 조각을 임베딩하여 '유사한' 텍스트를 찾는 데 특화되어 있습니다.¹⁷ 하지만 이는 "User 'A'의 '10km 완주'라는 목표와 '주 3회 조깅'이라는 루틴이 서로 연결되어 있다"는 **구조적, 관계적 컨텍스트(Relational Context)**를 저장하지 못합니다.

Neo4j와 같은 지식 그래프(Knowledge Graph) 데이터베이스는 벡터 검색 기능과 더불어 이러한 관계를 명시적으로 저장합니다.²⁰ 즉, 사용자의 데이터를 단순 텍스트가 아닌, **컨텍스트 그래프**로 저장해야 합니다 ²⁰:

- (User {id: 'A'}) -> (Goal {name: '10km 완주'})
- (Goal {name: '10km 완주'}) -> (Routine {name: '주 3회 조깅'})

Agentic GraphRAG²⁰ 접근 방식을 통해, '운동 전문가 에이전트'는 "User A의 모든 목표와 관련된 루틴을 가져와" (Cypher 쿼리) 같은 관계 기반의 심층 검색을 수행할 수 있습니다. 이는 단순 벡터 검색보다 훨씬 풍부하고 정확한 컨텍스트를 제공합니다.²⁰

또한, 귀사의 RunningRecord¹는 ****시계열 데이터(Time-Series Data)****입니다. 2025년 11월 최신 아키텍처는 이를 ****TKG(Temporal Knowledge Graph)****로 통합합니다.²³ 사용자의 러닝 기록이 발생할 때마다 Neo4j에 시간 정보와 함께 노드를 생성합니다:

- (User {id: 'A'}) -> (Run {distance: 5.2})

이는 Neo4j 내에 사용자의 이력(History)이 담긴 동적 메모리(**Dynamic Memory**)²³를 구축하며, 에이전트가 "사용자의 러닝 기록이 시간에 따라 어떻게 진화했는가"²³를 분석할 수 있게 합니다.

I.C. 시계열 데이터 처리: 특수 **RAG** 아키텍처 (**Time-Series RAG**)

RunningRecord¹와 같은 시계열 데이터를 처리하는 방식은 장기 기억의 핵심입니다.

RunningRecord의 모든 로그(수백만 건)를 청크(Chunk)로 쪼개 임베딩하고 벡터 DB에 저장하는 표준 RAG 방식은 잘못된 접근입니다.²⁵ 그 이유는 다음과 같습니다:

1. 의미의 차이: 데이터의 의미가 개별 데이터 포인트가 아닌 패턴, 트렌드, 상관관계에 있습니다.²⁵
2. 질문의 차이: 사용자는 '유사한 로그'가 아닌, "평균 페이스가 얼마인가?" 같은 분석적(**Analytical**) 질문을 합니다.²⁵
3. 데이터 볼륨: 원시 데이터를 모두 임베딩하는 것은 비효율적입니다.

따라서²⁵에서 제안하는 '**Query Parser** -> **TSDB Query** -> **Result Formatter**' 플로우를 채택해야 합니다.

주목할 점은, 귀사의 기존 **AIAnalysisDataCollector**¹는 이미 이 아키텍처를 개념적으로 구현하고 있다는 것입니다. `collectRunningData` 함수는 RunningRecord 원본을 그대로 LLM에 전달하는 것이 아니라, `statistics` (총 거리, 평균 페이스 등)와 `recentTrend` (증가/유지/감소)를 미리 계산합니다.¹

이것이 바로 '**TSDB Query** -> **Result Formatter**' (즉, 통계 계산) 단계입니다.²⁵ 따라서, 멀티 에이전트 시스템은 이 **DataCollector**의 로직을 폐기하는 것이 아니라, '운동 전문가 에이전트'가 호출할 수 있는 '**도구(Tool)**'로 리팩토링해야 합니다.

I.D. LLM 모델 매트릭스

1

¹ 1페이지의 단일 'GPT-5' 모델 접근 방식은 비용과 성능 면에서 비효율적입니다. 멀티 에이전트 시스템의 장점은 각 작업에 최적화된 (더 작고, 빠르고, 저렴한) 모델을 라우팅하는 데 있습니다.

- '운동 전문가'¹: 10개 섹션 분석, 통계 해석, 로드맵 생성을 위해 SOTA급 추론(Reasoning) 및 분석 능력이 필요합니다.
- '감성 코치'¹: 분석 결과를 바탕으로 감성적, 창의적, 공감적인 텍스트 생성이 필요합니다.²⁷
- '리더 에이전트'¹: 두 개의 구조화된 입력(전문가 분석, 코치 피드백)을 받아 요약 및 종합하는 능력이 필요합니다.

표 2: SOTA LLM 선정 매트릭스 (2025년 11월 기준)

에이전트 역할	핵심 요구 능력	권장 SOTA 모델 (2025년 11월)	근거 (Research)
1. 운동 전문가 에이전트	최고 수준의 추론 및 분석, 코딩(로드맵 생성).	Claude 4 Opus 또는 Gemini 2.5 Pro	Claude 4 Opus: 코딩(SWE-bench) 1위 ³⁰ , 복잡한 추론 및 문서 이해력. ³¹ Gemini 2.5 Pro: 복잡한 추론(GPQA) 1위. ³⁰
2. 감성 코치 에이전트	감성 지능, 공감 능력, 창의적/인간적 톤.	GPT-4o 또는 Claude 3.5 Sonnet	GPT-4o: 창의적 글쓰기, 실시간 상호작용, 다용도성. ³³ Claude 3.5 Sonnet: 뉘앙스 이해, 신뢰성 ³³ , GPT-4o 대비 높은 분류 정확도. ³⁴ (더 빠르고 저렴함)
3. 리더 에이전트	구조화된 입력 종합(Synthesis),	Claude 3.5 Sonnet (가장 권장)	2x 빠른 속도 ³³ , 강력한 요약/종합

	빠른 응답.		능력, 비용 효율성. ³³
--	--------	--	---------------------------

II. LangGraph 기반 멀티 에이전트 오케스트레이션 아키텍처

¹ 17페이지의 다이어그램과 19페이지의 MultiAgentOrchestrator를 LangGraph로 대체하는 실제 아키텍처를 설계합니다.

II.A. 신규 시스템 아키텍처 다이어그램 (LangGraph 기반)

새로운 아키텍처는 ¹ 17페이지의 개념적 흐름을 LangGraph의 구체적인 컴포넌트로 매핑합니다.

1. **** $\rightarrow$ **[LangGraph Orchestrator]** (app.stream 호출)
2. AgentState 초기화
3. **[Entry Point]** $\rightarrow$ ****
 - Tool Call $\rightarrow$ **** (Goals, Routines 조회)
 - Tool Call $\rightarrow$ **** (Running Stats 계산)
4. ****: route_after_data
 - **** $\rightarrow$ **[Node: Expert]**
 - **[Path 2: Fail (Validation)]** $\rightarrow$ **** $\rightarrow$ ****
5. **[Node: Expert]**
 - UI에 'Stage 1 완료' 이벤트 전송 ¹
6. ****: route_after_expert
 - **** $\rightarrow$ **[Node: Coach]**
 - **[Path 2: Fail]** $\rightarrow$ **[Node: Fallback Expert]** ¹
7. **[Node: Coach]**
 - UI에 'Stage 2 완료' 이벤트 전송 ¹
8. ****: route_after_coach
 - **** $\rightarrow$ **[Node: Leader]**
 - **[Path 2: Fail]** $\rightarrow$ **[Node: Fallback Coach]** ¹
9. **[Node: Leader]**
 - UI에 'Stage 3 완료' 이벤트 전송 ¹
10. _____

11. [LangGraph Orchestrator] $\rightarrow$ [UI] (최종 AgentState 반환)

II.B. 핵심: LangGraph 상태 머신 (AgentState) 설계

LangGraph의 모든 노드(에이전트)가 공유하는 중앙 메모리 객체¹⁴를 정의합니다. 이는 ¹18페이지의 'Sequential Data Cascade'를 코드로 구현한 것입니다. 이 **AgentState**는 귀사 시스템의 "혈류(Bloodstream)"이며, 각 에이전트는 이 상태를 읽고, 자신의 결과물을 이 상태에 추가합니다.

표 3: LangGraph AgentState 상세 정의 (Python TypedDict)

Python

```
from typing import TypedDict, List, Dict, Any, Optional
from typing_extensions import Annotated
import operator
from langchain_core.messages import AnyMessage
```

, p3의 'Collected Data Structure'를 반영

```
class UserData(TypedDict):
    userInfo: Dict[str, Any]
    goals: List  # GoalModel
    routines: List  # RoutineModel
    runningLogs: List  # RunningRecord
    statistics: Dict[str, Any]
```

, p15의 'Expert Agent Output Structure'를 반영

```
class ExpertAnalysis(TypedDict):
    patternEvaluation: str
    strengths: List[str]
    improvements: List[str]
    trendAnalysis: str
```

, p16의 'Coach Agent Output Structure'를 반영

```
class CoachFeedback(TypedDict):
    encouragement: str
    empathyMessage: str
```

```
motivationalAdvice: str
```

```
# , p16의 'Leader Agent Output Structure'를 반영
```

```
class LeaderResult(TypedDict):
```

```
    gapAnalysis: str
```

```
    weeklyRoadmap: List] # WeekPlan
```

```
    actionPlan: List] # Action
```

```
# 모든 에이전트가 공유할 중앙 상태
```

```
class AgentState(TypedDict):
```

```
    # 입력
```

```
    user_id: str
```

```
    category: str
```

```
# 데이터 수집 단계 (Node: DataCollector)
```

```
    user_data: Optional
```

```
# 1단계: 전문가 분석 (Node: Expert)
```

```
    expert_analysis: Optional[ExpertAnalysis]
```

```
# 2단계: 감성 코치 (Node: Coach)
```

```
    coach_feedback: Optional[CoachFeedback]
```

```
# 3단계: 리더 종합 (Node: Leader)
```

```
    final_result: Optional
```

```
# 시스템 상태
```

```
# 의 operator.add를 사용하여 메시지 기록을 누적
```

```
    messages: Annotated[List[AnyMessage], operator.add]
```

```
    error_message: Optional[str]
```

```
    fallback_used: bool
```

II.C. 워크플로우 정의: 노드(Node) 및 엣지(Edge)

¹ 19-21페이지의 `_execute...` 함수들을 LangGraph 노드로 매핑합니다. 각 노드는 AgentState를 입력받아, 자신의 작업을 수행한 후, 업데이트된 AgentState를 반환하는 Python 함수입니다.⁹

Python

```
from langgraph.graph import StateGraph, END
```

```
# 각 노드(에이전트) 함수 정의 (예: expert_node, coach_node 등)
```

```
def data_collection_node(state: AgentState) -> AgentState:
```

```
    #... 표 5의 도구들을 호출하여 state['user_data']를 채움...
```

```
    #... 검증 실패 시 state['error_message'] 설정...
```

```
    return state
```

```
def expert_node(state: AgentState) -> AgentState:
```

```
    #... state['user_data']를 기반으로 LLM 호출 (표 2 모델)...
```

```
    #... state['expert_analysis']를 채움...
```

```
    #... 실패 시 state['error_message'] 설정...
```

```
    return state
```

```
def coach_node(state: AgentState) -> AgentState:
```

```
    #... state['expert_analysis']를 기반으로 LLM 호출...
```

```
    #... state['coach_feedback']를 채움...
```

```
    return state
```

```
def leader_node(state: AgentState) -> AgentState:
```

```
    #... state['user_data'], state['expert_analysis'], state['coach_feedback'] 종합...
```

```
    #... state['final_result']를 채움...
```

```
    return state
```

```
# Fallback 노드 정의
```

```
def fallback_expert_node(state: AgentState) -> AgentState:
```

```
    #... state['user_data']['statistics'] 기반 기본 분석 생성...
```

```
    state["expert_analysis"] = {
```

```
        "patternEvaluation": "데이터 분석 중 일시적 오류 발생. 기본 통계입니다.",
```

```
        "strengths": [f"총 {state['user_data']['statistics']}일 러닝 완료"],
```

```
        "improvements": ["AI 상세 분석 실패. 잠시 후 다시 시도해주세요."],
```

```
        "trendAnalysis": f"최근 트렌드: {state['user_data']['statistics']}"
```

```
    }
```

```
    state["fallback_used"] = True
```

```
    return state
```

```
#... (기타 노드 정의)...
```

```
workflow = StateGraph(AgentState)
```

```
# 1. 노드 등록
workflow.add_node("collect_data", data_collection_node)
workflow.add_node("expert", expert_node)
workflow.add_node("coach", coach_node)
workflow.add_node("leader", leader_node)
workflow.add_node("fallback_expert", fallback_expert_node)
#... (기타 fallback 노드 등록)...
```

```
# 2. 진입점 설정
workflow.set_entry_point("collect_data")
```

```
# 3. 엣지 연결 (II.D에서 상세화)
workflow.add_edge("leader", END)
```

II.D. Fallback 구현: 조건부 엣지(Conditional Edges)

이것이 LangGraph를 선택한 핵심 이유입니다.¹ 24-25페이지의 'Fallback Strategy'를 구현합니다.¹⁰ `add_edge` 대신 `add_conditional_edges`를 사용하여 동적 라우팅을 구현합니다.

Python

```
# 1. 라우터 함수 정의 [11]
def route_after_data(state: AgentState) -> str:
    # , p4의 Validation 로직이 여기 포함
    if state.get("error_message"): # 데이터 검증/수집 실패 시
        return "end_with_error" # (별도 정의된 에러 종료 노드)
    else:
        return "expert"

def route_after_expert(state: AgentState) -> str:
    # , p24의 로직 구현
    if state.get("error_message") and not state.get("fallback_used"):
        return "fallback_expert" # 실패 시 Fallback 노드로
    else:
        return "coach" # 성공 시 Coach 노드로

def route_after_coach(state: AgentState) -> str:
```

```
# , p24의 로직 구현
if state.get("error_message") and not state.get("fallback_used"):
    return "fallback_coach"
else:
    return "leader"
```

2. 조건부 엣지 등록

```
workflow.add_conditional_edges(
    "collect_data",
    route_after_data,
    {
        "expert": "expert",
        "end_with_error": END
    }
)
```

```
workflow.add_conditional_edges(
    "expert",
    route_after_expert,
    {
        "coach": "coach",
        "fallback_expert": "fallback_expert"
    }
)
```

, p24 시나리오: "Expert fails -> Show basic stats + generic coach + basic plan"

즉, fallback_expert 노드는 coach가 아닌 fallback_coach로 가야 함

```
workflow.add_edge("fallback_expert", "fallback_coach") # (fallback_coach 노드는 generic 응답)
```

```
workflow.add_conditional_edges(
    "coach",
    route_after_coach,
    {
        "leader": "leader",
        "fallback_coach": "fallback_coach"
    }
)
```

Fallback Coach는 Leader로 가야 함 (Leader는 Fallback 인지하고 기본 로드맵 생성)

```
workflow.add_edge("fallback_coach", "leader")
```

III. 하이브리드 RAG 및 장기 기억 시스템 구현

에이전트가 사용할 데이터를 저장하고 검색하는 백엔드 아키텍처입니다.

III.A. Neo4j 스키마 설계: 사용자의 '컨텍스트 그래프'

¹ 12페이지의 데이터 모델을 Neo4j 스키마 ²⁰로 변환합니다.

표 4: Neo4j 지식 그래프 스키마 (Cypher)

항목	Cypher (예시)	설명
노드 (Nodes)	CREATE (u:User {id: '...', name: '...'})	GlobalUser
	CREATE (g:Goal {id: '...', name: '10km 완주', category: '운동'})	GoalModel
	CREATE (r:Routine {id: '...', name: '아침 조깅', category: '운동'})	RoutineModel
	CREATE (rec:RunningRecord {id: '...', date: '...', distance: 5.2})	RunningRecord (TKG) ²³
관계 (Edges)	MATCH (u:User), (g:Goal) WHERE u.id = '...' AND g.id = '...' CREATE (u)-->(g)	사용자가 목표를 소유
	MATCH (g:Goal), (r:Routine) WHERE... CREATE (g)-->(r)	목표와 루틴의 연관성

	MATCH (u:User), (rec:RunningRecord) WHERE... CREATE (u)-->(rec)	사용자의 TKG (시계열) ²³
인덱스	CREATE CONSTRAINT user_id IF NOT EXISTS FOR (u:User) REQUIRE u.id IS UNIQUE	빠른 사용자 조회를 위한 필수 인덱스

III.B. 시계열 데이터 **RAG: RunningRecord** 처리 및 도구 리팩토링

I.C에서 분석했듯이, RunningRecord ¹는 특수 RAG ²⁵가 필요하며, ¹ 3페이지의 AIAnalysisDataCollector 로직을 **에이전트 도구(Agent Tools)**로 리팩토링해야 합니다.³⁶

RunningRecord 원본 데이터는 PostgreSQL (pgvector와 호환 ¹⁸) 또는 TimescaleDB (시계열 전문)에 저장합니다. data_collection_node는 다음 도구들을 호출합니다.

표 5: 'DataCollector 에이전트' 도구 명세서 ¹

도구 (Python 함수)	설명	입력	출력
@tool get_user_context	Neo4j에서 사용자의 프로필(GlobalUser), 목표(GoalModel), 루틴(RoutineModel)을 조회합니다. ²⁰	user_id: str	dict (userInfo, goals, routines)
@tool get_running_statistics	RunningRecord DB(TSDB/PG)에서 최근 N일간의 통계 (총 일수, 총 거리, 평균 페이스 등)를 계산합니다. ¹	user_id: str, days: int = 60	dict (statistics)

@tool get_recent_running_logs	RunningRecord DB에서 최근 N개의 상세 로그를 조회합니다. ¹	user_id: str, limit: int = 10	List
@tool get_recent_trend	RunningRecord DB에서 최근 7개 기록 vs 이전 7개 기록을 비교하여 트렌드(증가/유지/감소)를 분석합니다. ¹	user_id: str	str ("증가", "유지", "감소")
@tool validate_data	수집된 데이터가 분석에 충분한지 검증합니다. ¹	data: UserData	(bool, List[str]) (유효성, 실패 사유)

IV. 단계별 에이전트 구현 및 통합 가이드 (Phase 1-4)

¹ 25-27페이지의 'Transition Roadmap'을 기반으로, 위에서 정의한 LangGraph 및 Neo4j 스택을 사용한 실제 구현 로드맵을 제시합니다.

IV.A. Phase 1: '운동 전문가 에이전트' 구현 및 도구 연동

1

시스템에서 가장 복잡한 노드로, 데이터 분석가 역할을 수행합니다.³⁸

1. data_collection_node 구현:

이 노드는 AgentState의 user_id를 받아, [표 5]의 도구들을 순차적으로 호출하여 state['user_data']를 채웁니다. validate_data 도구 호출 후, 검증 실패 시 state['error_message']를 설정하고, route_after_data 라우터(II.D)가 이를 감지하여 워크플로우를 조기 종료시킵니다.

2. expert_node 구현:

이 노드는 state['user_data']가 이미 채워져 있다고 가정하고, 이 데이터를 LLM([표 2]의 Claude 4 Opus 등)에 전달하여 ****분석(Analysis)****에만 집중합니다. LLM이 115페이지의 구조화된 JSON 출력을 생성하도록 유도하는 것이 중요합니다.

3. '운동 전문가 에이전트' 시스템 프롬프트 1:

기존 프롬프트1는 '15년 경력 코치'라는 역할만 정의합니다. 새로운 프롬프트는 LLM에게 입력 데이터 형식과 출력 형식을 명확히 지시해야 합니다.40

XML

당신은 15년 경력의 프로 러닝 코치이며, 스포츠 과학 석사 학위를 보유한 데이터 분석 전문가입니다.

다음은 <user_data> 태그에 제공된 러너의 상세 데이터입니다.

```
<user_data>
{state.user_data}
</user_data>
```

당신의 임무는 이 데이터를 심층 분석하여 다음 4가지 섹션으로 구성된 전문적인 분석 리포트를 생성하는 것입니다.

분석은 반드시 <user_data>의 수치(통계, 목표, 신체 정보)에 기반해야 합니다.

반드시 다음 JSON 스키마를 준수하여 응답해야 합니다. 다른 텍스트는 절대 추가하지 마십시오.

```
{
  "patternEvaluation": "",
  "strengths": [
    "",
    ""
  ],
  "improvements": [
    "",
    ""
  ],
  "trendAnalysis": ""
}
```

이 노드는 LLM의 JSON 출력을 파싱하여 state['expert_analysis']에 저장합니다. 파싱 실패 시 state['error_message']를 설정합니다.

IV.B. Phase 2: '감성 코치' 및 '리더 에이전트' 구현

1

이 노드들은 expert_node의 출력을 입력받아 순차적으로 작동합니다.

1. coach_node 구현:

이 에이전트는 state['expert_analysis']를 입력으로 받습니다. 이 에이전트는 분석을 수행하지 않고, expert_analysis.improvements (개선점)를 감성적으로 다시 말하는(Rephrasing) 역할에 집중합니다.²⁷

'감성 코치 에이전트' 시스템 프롬프트 ¹:

XML

당신은 따뜻하고 공감 능력이 뛰어난 러닝 코치입니다.
전문가 분석 결과가 <expert_analysis> 태그로 주어집니다.

```
<expert_analysis>
{state.expert_analysis}
</expert_analysis>
```

사용자는 '도전적 성향'입니다.

전문가 분석의 <improvements> 섹션(개선점)을 사용자가 부담스럽지 않고 긍정적으로 받아들일 수 있도록,

따뜻한 격려와 동기부여 메시지로 바꾸어 주세요.

"이것만 보완하면 훨씬 더 성장할 수 있다"는 톤을 유지하세요.

다음 JSON 형식을 준수하여 응답하세요.

```
{
  "encouragement": "을 기반으로 한 따뜻한 응원 메시지)",
  "empathyMessage": "에 대한 공감적 피드백)",
  "motivationalAdvice": ""
}
```

결과를 state['coach_feedback']에 저장합니다.

2. leader_node 구현:

이 에이전트는 state['user_data'], state['expert_analysis'], state['coach_feedback']을 모두 입력받아 최종 계획을 종합합니다.

'리더 에이전트' 시스템 프롬프트 ¹:

XML

당신은 전문가 분석과 감성 피드백을 종합하는 리더 에이전트입니다.

<user_data>, <expert_analysis>, <coach_feedback> 태그의 모든 정보를 종합하세요.

```
<user_data>
{state.user_data}
</user_data>
<expert_analysis>
{state.expert_analysis}
</expert_analysis>
<coach_feedback>
{state.coach_feedback}
</coach_feedback>
```

이제, <user_data>의 목표와 <expert_analysis>의 분석 결과를 바탕으로 다음 2가지 최종 결과물을 생성하세요:

1. 4주간의 단계별 목표 달성 로드맵
2. 다음 주 실천 계획 (Top 5 우선순위)

(만약 state['fallback_used']가 True라면, 'weeklyRoadmap'과 'actionPlan' 대신 기본 권장 사항(예: '주 3회 러닝 유지')을 생성해야 함)

다음 JSON 형식을 준수하여 응답하세요.

```
{
  "gapAnalysis": "과 목표('goals') 간의 Gap을 구체적으로 분석)",
  "weeklyRoadmap": [
    {"week": 1, "distance": "...", "pace": "...", "frequency": "..."},
    {"week": 2, "distance": "...", "pace": "...", "frequency": "..."},
    {"week": 3, "distance": "...", "pace": "...", "frequency": "..."},
    {"week": 4, "distance": "...", "pace": "...", "frequency": "..."}
  ],
  "actionPlan": [
    {"priority": 1, "action": ""},
    {"priority": 2, "action": "..."},
    {"priority": 3, "action": "..."},
    {"priority": 4, "action": "..."},
    {"priority": 5, "action": "..."}
  ]
}
```

결과를 state['final_result']에 저장합니다.

IV.C. Phase 3: MultiAgentOrchestrator 재구축 및 UI 연동

1

¹ 19페이지의 개념적 MultiAgentOrchestrator 클래스와 23페이지의 'Progressive 3-Stage Display'를 LangGraph로 구현합니다. ¹ 19페이지의 performAnalysis 함수는 LangGraph의 app.stream() 또는 app.astream_events() ¹ 호출로 대체됩니다.

1. LangGraph app 컴파일:

Python

II.C와 II.D에서 정의한 workflow 객체를 컴파일

checkpointer를 추가하여 상태 저장 및 재시작이 가능하도록 함 [9, 16]

```
from langgraph.checkpoint.sqlite import SqliteSaver
```

```
memory = SqliteSaver.from_conn_string(":memory:") # 프로덕션에서는 영구 DB 사용  
app = workflow.compile(checkpointer=memory)
```

2. UI 연동: onProgress 콜백 구현 1:

AiAnalysisScreen 1은 백엔드(FastAPI 등)의 스트리밍 엔드포인트를 호출해야 합니다. 이 엔드포인트는 app.astream_events()를 구독하여 각 노드의 완료 이벤트를 실시간으로 UI(Flutter)에 전송합니다.

Python

백엔드(FastAPI)의 스트리밍 엔드포인트 (예시)

```
@app.post("/stream-analysis/")
```

```
async def stream_analysis(request: AnalysisRequest):
```

```
    # user_id로 고유한 스레드 ID 생성
```

```
    config = {"configurable": {"thread_id":
```

```
f"user_{request.user_id}_{datetime.now().isoformat()}"}}
```

```
    initial_state = AgentState(
```

```
        user_id=request.user_id,
```

```
        category=request.category,
```

```
        messages=,
```

```
        fallback_used=False
```

```
)
```

```
    async def event_stream():
```

```
        # astream_events는 각 노드(체인)의 시작/끝 이벤트를 스트리밍함
```

```
    async for event in app.astream_events(initial_state, config, version="v1"):
```

```
        event_type = event["event"]
```

```
        event_name = event.get("name")
```

```
    # , p23의 'Progressive Display' 로직 매핑
```

```
    if event_type == "on_chain_end" and event_name == "expert":
```

```
        # onProgress('감성 피드백 생성 중...', 0.6)
```

```
        output = event["data"]["output"]
```

```
        yield json.dumps({
```

```
            "message": "감성 피드백 생성 중...",
```

```
            "progress": 0.6,
```

```
            "stage": 2,
```

```
            "data": output.get("expert_analysis") # 1단계 결과 전송
```

```
        }) + "\n"
```

```
    elif event_type == "on_chain_end" and event_name == "coach":
```

```
        # onProgress('최종 결과 종합 중...', 0.8)
```

```
        output = event["data"]["output"]
```

```

        yield json.dumps({
            "message": "최종 결과 종합 중...",
            "progress": 0.8,
            "stage": 3,
            "data": output.get("coach_feedback") # 2단계 결과 전송
        }) + "\n"

#... (data_collection_node 완료 시 0.2 전송 등)...

elif event_type == "on_chain_end" and event.get("name") == "__main__":
    # 그래프 전체 실행 종료
    output = event["data"]["output"]
    yield json.dumps({
        "message": "완료!",
        "progress": 1.0,
        "stage": 4,
        "data": output.get("final_result") # 최종 결과 전송
    }) + "\n"

return StreamingResponse(event_stream(), media_type="text/event-stream")

```

IV.D. Phase 4: 테스트, 최적화 및 배포

1

- 테스트:¹ 24페이지의 'Fallback Strategy'를 테스트하기 위해, **expert_node**가 의도적으로 실패하도록 하는(예: `validate_data` 도구가 항상 **False**를 반환) 유닛 테스트를 작성해야 합니다. **LangGraph**는 각 노드를 개별적으로 테스트하기 용이합니다.
- 모니터링 및 최적화:
 - **LangGraph**는 **LangChain** 생태계에 속하므로, **LangSmith**¹⁶를 사용하여 각 노드의 지연 시간, 토큰 사용량, 비용을 추적하는 것이 강력히 권장됩니다.
 - 이는¹ 22페이지의 'Performance Comparison'에 명시된 ****핵심 성과 지표(KPI)****를 정확히 측정하고 달성하는 데 필수적입니다.
 - 목표 달성¹:
 - 지연 시간 (**< 10초**): [표 2]의 모델 라우팅(예: Coach/Leader에 빠른 Claude 3.5 Sonnet 사용)으로 달성.
 - 토큰 비용 (**< 2,800**): 단일 GPT-5¹ 대신 작업에 최적화된 소형 모델을 혼용하여 달성.
 - 에러율 (**< 5%**): II.D에서 설계한 'Agent-Level Fallback' 로직으로 안정성 확보.

V. 결론 및 최종 제안

귀사가¹에서 제시한 3-에이전트 아키텍처는 세르파엥의 AI 코칭 기능을 혁신할 수 있는 매우 정확한 방향입니다. 현재의 단일 LLM 호출 방식¹은 귀사가 이미 파악한 바와 같이¹ 확장성과 사용자 경험에 명확한 한계가 있습니다.

본 보고서는 귀사의 이 탁월한 **개념적 설계¹**를 2025년 11월 프로덕션 환경에 구현하기 위한 기술적 청사진을 제시했습니다.

핵심 권장 사항 요약:

1. 오케스트레이션: **LangGraph**를 채택하십시오.² 이는 귀사의 핵심 요구 사항인 'Agent-Level Fallback'¹과 'Progressive UX'¹를 '조건부 엣지'¹⁰와 '이벤트 스트리밍'¹⁴으로 완벽하게 지원하는 2025년 SOTA 프레임워크입니다.
2. 장기 기억: **Neo4j**를 도입하십시오.²⁰ 단순 벡터 DB¹⁸를 넘어, GoalModel과 RoutineModel¹ 간의 관계를 저장하는 '컨텍스트 그래프'²²와 RunningRecord의 이력을 추적하는 'TKG'²³를 구현해야 합니다.
3. 데이터 처리: 리팩토링을 진행하십시오. 귀사의 AIAnalysisDataCollector¹ 로직은 'Time-Series RAG'²⁵의 핵심입니다. 이를 모놀리식 함수에서 'DataCollector 에이전트'가 사용하는 @tool 세트([표 5])로 리팩토링해야 합니다.³⁷
4. 모델: 라우팅을 적용하십시오. 단일 'GPT-5'¹ 대신, 각 에이전트의 작업(분석, 공감, 종합)에 최적화된 SOTA LLM([표 2])을 사용하여 비용과 성능을 모두 최적화하십시오.

이 청사진을 따라 귀사의 Transition Roadmap¹을 실행한다면, 현재 15-20초의 '블랙박스' 시스템을 7-10초 내에 신뢰할 수 있는 피드백을 점진적으로 제공하는¹ SOTA급 AI 코칭 시스템으로 성공적으로 전환할 수 있을 것입니다.

참고 자료

1. AI_ANALYSIS_SYSTEM_ARCHITECTURE.pdf
2. LangGraph vs CrewAI: Let's Learn About the Differences - ZenML Blog, 11월 12, 2025에 액세스, <https://www.zenml.io/blog/langgraph-vs-crewai>
3. LangGraph vs AutoGen vs CrewAI: Complete AI Agent Framework Comparison + Architecture Analysis 2025 - Latenode, 11월 12, 2025에 액세스, <https://latenode.com/blog/platform-comparisons-alternatives/automation-platform-comparisons/langgraph-vs-autogen-vs-crewai-complete-ai-agent-framework-comparison-architecture-analysis-2025>
4. AutoGen vs. CrewAI vs. LangGraph vs. OpenAI Multi-Agents Framework - Galileo AI, 11월 12, 2025에 액세스,

<https://galileo.ai/blog/autogen-vs-crewai-vs-langgraph-vs-openai-agents-framework>

5. Top AI Agent Frameworks in 2025 - Codecademy, 11월 12, 2025에 액세스, <https://www.codecademy.com/article/top-ai-agent-frameworks-in-2025>
6. Crewai vs. LangGraph: Multi agent framework comparison - Zams, 11월 12, 2025에 액세스, <https://www.zams.com/blog/crewai-vs-langgraph>
7. CrewAI vs LangGraph vs n8n | AI Agent Framework Comparison - 3Pillar Global, 11월 12, 2025에 액세스, <https://www.3pillarglobal.com/insights/blog/comparison-crewai-langgraph-n8n/>
8. AutoGen vs CrewAI: Which Multi-Agent Framework Should You Build With in 2025? - Sider, 11월 12, 2025에 액세스, <https://sider.ai/blog/ai-tools/autogen-vs-crewai-which-multi-agent-framework-should-you-build-with-in-2025>
9. How to Build LangGraph Agents Hands-On Tutorial | DataCamp, 11월 12, 2025에 액세스, <https://www.datacamp.com/tutorial/langgraph-agents>
10. Graph API overview - Docs by LangChain, 11월 12, 2025에 액세스, <https://docs.langchain.com/oss/python/langgraph/graph-api>
11. A Comprehensive Guide to LangGraph: Managing Agent State with Tools - Medium, 11월 12, 2025에 액세스, <https://medium.com/@o39joey/a-comprehensive-guide-to-langgraph-managing-agent-state-with-tools-ae932206c7d7>
12. Advanced LangGraph: Implementing Conditional Edges and Tool-Calling Agents, 11월 12, 2025에 액세스, <https://dev.to/jamesli/advanced-langgraph-implementing-conditional-edges-and-tool-calling-agents-3pdn>
13. Trustworthy Agents with LangGraph and Cleanlab, 11월 12, 2025에 액세스, https://help.cleanlab.ai/tlm/use-cases/tlm_agents/
14. Quickstart - Docs by LangChain, 11월 12, 2025에 액세스, <https://docs.langchain.com/oss/python/langgraph/quickstart>
15. Building a Multi-Agent AI with LangGraph: A Comprehensive Guide - DEV Community, 11월 12, 2025에 액세스, <https://dev.to/hulk-pham/building-a-multi-agent-ai-with-langgraph-a-comprehensive-guide-57nj>
16. LangGraph - LangChain, 11월 12, 2025에 액세스, <https://www.langchain.com/langgraph>
17. Best Vector Databases in 2025: A Complete Comparison Guide - Firecrawl, 11월 12, 2025에 액세스, <https://www.firecrawl.dev/blog/best-vector-databases-2025>
18. Best 17 Vector Databases for 2025 [Top Picks] - lakeFS, 11월 12, 2025에 액세스, <https://lakefs.io/blog/best-vector-databases/>
19. Best Vector Databases for RAG: Complete 2025 Comparison Guide - Latenode, 11월 12, 2025에 액세스, <https://latenode.com/blog/best-vector-databases-for-rag-complete-2025-comparison-guide>
20. AI Systems - Build Smarter AI Systems Faster With Context - Neo4j, 11월 12, 2025에 액세스, <https://neo4j.com/use-cases/ai-systems/>

21. Building a GraphRAG Agent with Neo4j and Milvus, 11월 12, 2025에 액세스,
<https://neo4j.com/blog/developer/graphrag-agent-neo4j-milvus/>
22. Building AI Agents With the Google Gen AI Toolbox and Neo4j Knowledge Graphs - Medium, 11월 12, 2025에 액세스,
<https://medium.com/neo4j/building-ai-agents-with-the-google-gen-ai-toolbox-and-neo4j-knowledge-graphs-86526659b46a>
23. Building Evolving AI Agents Via Dynamic Memory Representations Using Temporal Knowledge Graphs - Neo4j, 11월 12, 2025에 액세스,
<https://neo4j.com/nodes-2025/agenda/building-evolving-ai-agents-via-dynamic-memory-representations-using-temporal-knowledge-graphs/>
24. Graphiti: Knowledge Graph Memory for an Agentic World - Neo4j, 11월 12, 2025에 액세스,
<https://neo4j.com/blog/developer/graphiti-knowledge-graph-memory/>
25. Build AI Monitoring RAG App | Why Time-Series Data Needs a ..., 11월 12, 2025에 액세스,
<https://vardhmanandroid2015.medium.com/build-ai-monitoring-rag-app-why-time-series-data-needs-a-different-approach-vs-8eeaca8a03f5>
26. Build AI Monitoring RAG App | Why Time-Series Data Needs a DIFFERENT Approach (vs. Docs, DBs, API) - YouTube, 11월 12, 2025에 액세스,
https://www.youtube.com/watch?v=xv3ahisu_UU
27. ElevenLabs Agents prompting guide, 11월 12, 2025에 액세스,
<https://elevenlabs.io/docs/agents-platform/best-practices/prompting-guide>
28. Use Emotion Prompting to Improve AI Interactions, 11월 12, 2025에 액세스,
<https://relevanceai.com/prompt-engineering/use-emotion-prompting-to-improve-ai-interactions>
29. AI with personality — Prompting ChatGPT using BIG FIVE values | by Andrei Damsa, 11월 12, 2025에 액세스,
<https://medium.com/@damsa.andrei/ai-with-personality-prompting-chatgpt-using-big-five-values-def7f050462a>
30. 10 Best LLMs of November 2025: Performance, Pricing & Use Cases - Azumo, 11월 12, 2025에 액세스,
<https://azumo.com/artificial-intelligence/ai-insights/top-10-llms-0625>
31. Claude 3.5 Sonnet vs GPT 4o: Model Comparison 2025 - Galileo AI, 11월 12, 2025에 액세스,
<https://galileo.ai/blog/claude-3-5-sonnet-vs-gpt-4o-enterprise-ai-model-comparison>
32. Best 44 Large Language Models (LLMs) in 2025 - Exploding Topics, 11월 12, 2025에 액세스, <https://explodingtopics.com/blog/list-of-llms>
33. Claude 3.5 vs GPT-4o: Comparing Top AI Models in 2025 - Kanerika, 11월 12, 2025에 액세스, <https://kanerika.com/blogs/claude-3-5-vs-gpt-4o/>
34. Comparison Analysis: Claude 3.5 Sonnet vs GPT-4o - Vellum AI, 11월 12, 2025에 액세스, <https://www.vellum.ai/blog/claude-3-5-sonnet-vs-gpt4o>
35. Claude 3.5 Sonnet vs GPT-4o: Complete AI Model Comparison - SentiSight.ai, 11월 12, 2025에 액세스,
<https://www.sentisight.ai/claude-3-5-sonnet-vs-gpt-4o-ultimate-comparison/>

36. Building an Interactive AI Agent for Lightning-Fast Machine Learning ..., 11월 12, 2025에 액세스, <https://developer.nvidia.com/blog/building-an-interactive-ai-agent-for-lightning-fast-machine-learning-tasks/>
37. Building a Data Analysis AI Agent for Predictive Analytics - Thesys, 11월 12, 2025에 액세스, <https://www.thesys.dev/blogs/create-ai-agent-for-data-analysis>
38. Building a Data Analyst AI Agent using OpenAI Function Calling, 11월 12, 2025에 액세스, <https://www.analyticsvidhya.com/blog/2025/08/building-data-analyst-ai-agent/>
39. Build a Data Analyst AI Agent from Scratch | by Daniel Herrera | Teradata | Medium, 11월 12, 2025에 액세스, <https://medium.com/teradata/build-a-data-analyst-ai-agent-from-scratch-f613a1f5027c>
40. Master Prompt Engineering in 30 min (Full Course) - YouTube, 11월 12, 2025에 액세스, <https://www.youtube.com/watch?v=nfalAg8dFRl>
41. Develop an AI Coach with Effective Prompt Engineering Techniques - Arsturn, 11월 12, 2025에 액세스, <https://www.arsturn.com/blog/developing-an-ai-coach-prompt-engineering-for-personalized-guidance>
42. How to Build a Feedback-Driven AI Agent Using LangGraph (Step-by-Step Tutorial), 11월 12, 2025에 액세스, <https://medium.com/@jaslearns/how-to-build-a-feedback-driven-ai-agent-using-langgraph-step-by-step-tutorial-e5ed2c01d544>
43. LangGraph and Research Agents - Pinecone, 11월 12, 2025에 액세스, <https://www.pinecone.io/learn/langgraph-research-agent/>